> **Version:** 1.0 — August 2026
> **Scope:** End-to-end data flow from user query to LLM response
> **Source files:** `backend/app/agents/`, `backend/app/ai/`, `backend/app/routers/agents.py`

## e of Contents



## ecutive Summary

om's chat engine is a **tool-calling agent architecture**, not a vector-based RAG system. There are no embeddings, no vector Chroma/FAISS/Pinecone), and no LangChain. Instead:

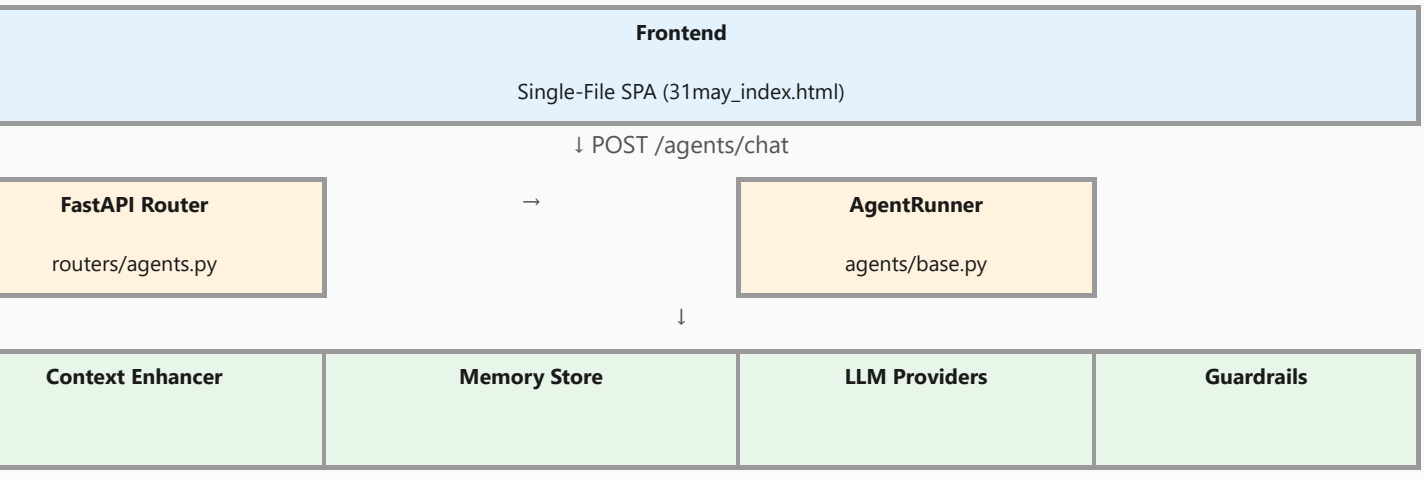
The **LLM** receives a system prompt containing module-specific instructions and live organizational data.
The LLM issues `[TOOL_CALL:tool_name:args]` markers in its response.
The **server** parses these markers, executes MySQL queries via tool functions, and feeds the results back to the LLM.
The LLM then produces a natural-language summary of the tool results for the user.

**erm context** is provided by a MySQL-backed memory system (`ai_memory` table) that stores confidence-scored insights from past ations, serving a similar role to vector retrieval but via SQL rather than similarity search.

## chitecture Overview Diagram

### gh-Level System Architecture

**re 1 — High-Level System Architecture**



| | | |
|---|---|---|
| | | |
| | | |
| | | |

```
ai_memory
 org_id = %s AND agent_name = %s
user_id IS NULL OR user_id != %s)
 BY confidence DESC, accessed_at DESC LIMIT %s
```

up to 8 memories ranked by confidence then recency.

**B — Recent conversation summaries** (`ai/query_enhancer.py:64-103`):

```
t up to 3 most recent conversations for this agent+user
T c.id, c.title, c.created_at
agent_conversations c
 c.agent_name = %s AND c.user_id = %s
 BY c.created_at DESC LIMIT 3

r each conversation, get last 3 messages
T role, content FROM agent_messages
 conversation_id = %s ORDER BY id DESC LIMIT 3
```

es are truncated to 150 chars each and formatted as `User: ... | Agent: ...`.

**C — Org context hint:**

```
ONTEXT ---
ization ID: {org_id} | Agent: {agent_name}
```

## — System Prompt Assembly

`nts/base.py:73-76`

l system prompt is assembled in three parts:

```
T_PROMPTS[agent_name]}           ← Step 4a: Agent-specific persona + tool docs
enhanced}                         ← Step 4b: Memories + recent convos + org hint
-- CURRENT ORGANIZATION DATA ---\n{context}  ← Step 4c: Live module data
```

loads from `AGENT_PROMPTS` (`agents/prompts.py:1-346`), which defines 12 agent personas. Each prompt includes: - Role description abilities - Available tools with `[TOOL_CALL:...]` syntax documentation - Response formatting instructions

truncates context to `MAX_PROMPT_LENGTH` (default: **50,000** characters) if it exceeds the limit (`base.py:57-58`).

## — Conversation History Load

`nts/base.py:79-81`

```
nversation_id:
istory = await self._load_history(conversation_id)
essages = history
```

he last **20 messages** from `agent_messages` via the Java DB service HTTP endpoint (`base.py:729-735`). This gives the LLM continuity urns.

## — Intent Auto-Detection & Query Rewriting

```
tManagementList → SELECT ... FROM audit_management
ieveTaskList/   → SELECT ... FROM task_details
ingManagementList/ → SELECT ... FROM meeting_management
liance          → SELECT ... FROM compliance_details
eCardList       → SELECT ... FROM score_card
ByUser          → SELECT ... FROM employee_details WHERE email = ?
oyeeDetailsList → SELECT ... FROM employee_details
List            → SELECT ... FROM swot_details
elList          → SELECT ... FROM pestel_details
```

n paths fall through to HTTP calls to the Java microservices (ports 9010-9060).

### AC Filtering Patterns

functions implement **role-based access control**:

**users** see all data across orgs (unscoped queries):

```
min task query (no WHERE clause on org)
T t.ID, t.task_value, t.owner, t.priority, t.status
task_details t
 BY FIELD(t.priority, 'Critical', 'High', 'Medium', 'Low'), t.ID
```

**r users** are scoped via `employee_details` JOIN:

```
mber task query (org + ownership scoped)
T t.ID, t.task_value, t.owner, t.priority, t.status
task_details t
employee_details e ON e.emp_id = t.owner
 e.org_id = %s AND t.owner = %s
 BY FIELD(t.priority, 'Critical', 'High', 'Medium', 'Low'), t.ID
```

**y resolution** always starts with email → emp_id lookup:

```
T emp_id, org_id FROM employee_details
 LOWER(email_address) = LOWER(%s) LIMIT 1
```

**rd RBAC** adds an additional `assigned_user_id` filter for non-admin, non-manager users (`scorecard_tools.py:60-61`).

### ON Blob Pattern

ySQL tables store structured data in a JSON column (e.g., `task_value`, `risk_value`, `initiative_value`, `incident_value`, `n_value`). The bridge parses these via `bridge._parse_json_col()`. When updating, the full JSON blob is read, merged with changes, tten back.

---

## ntext & Prompt Assembly

### gent System Prompts

`nts/prompts.py:1-346`

t personas are defined in the `AGENT_PROMPTS` dictionary:

```
URRENT ORGANIZATION DATA ===
ISK AGENT DATA ===

ISKS ---
 | risk_value={"name":"Cyber Breach","score":"20",...} | owner=101 | status=APPROVED
 | risk_value={"name":"Vendor Lock-in","score":"15",...} | owner=105 | status=APPROVED


UDIT ---
 | managementvalue={...} | owner=101 | active=1


OMPLIANCE ---
 | complain_value={...} | status=Compliant | risklevel=Low
```

### ken & Character Limits

`e/config.py:46-56`

| eter | Default | Purpose |
|---|---|---|
| `HAT_INPUT_CHARS` | 8,000 | Max user message length |
| `ROMPT_LENGTH` | 50,000 | Max total system prompt (context block) |
| `ESPONSE_CHARS` | 16,000 | Hard output cap on LLM response |
| `LM_MAX_TOKENS` | 2,048 | Provider `max_tokens` ceiling |
| `OOL_RESULT_CHARS` | 4,000 | Max tool result fed back to LLM |

## ol-Calling Mechanism

### ol Call Protocol

M emits tool calls as inline text markers:

```
_CALL:query_risks:min_heat=10]
_CALL:create_task:title=Deploy MFA,owner=dominic@demo.com,priority=High]
_CALL:update_task_status:id=102,status=completed]
```

: `[TOOL_CALL:{tool_name}:{key1}={value1},{key2}={value2}]`

ex parser (`base.py:31`):

```
_CALL_RE = re.compile(r"\[TOOL_CALL:([^\]:]+)(?::([^\]]*))?\]")
```

Group 1: tool name (e.g., `query_risks`)
Group 2: optional args string (e.g., `min_heat=10`)

### gument Parsing

`ents/base.py:395-412` (`_parse_kv`)

```
            pass
        result[k.strip()] = v.strip() if isinstance(v, str) else v
eturn result
```

## mplete Tool Inventory

| lame | File | MySQL Table | Operation |
|---|---|---|---|
| _tasks | task_tools.py:87 | task_details | SELECT (with RBAC) |
| e_task_status | task_tools.py:276 | task_details | UPDATE (status column + JSON) |
| e_task_progress | task_tools.py:424 | task_details | UPDATE (progress in JSON, auto-status) |
| e_task | task_tools.py:574 | task_details | INSERT |
| e_risk_mitigation_task | task_tools.py:698 | task_details + risk_details | INSERT (linked task) |
| _risks | risk_tools.py:22 | risk_details | SELECT (with RBAC) |
| e_risk | risk_tools.py:146 | risk_details | INSERT |
| e_risk | risk_tools.py:299 | risk_details | UPDATE (JSON blob merge) |
| e_risk | risk_tools.py:497 | risk_details | DELETE (admin only) |
| simulator | risk_tools.py:588 | risk_details | SELECT → Monte Carlo simulation |
| _scorecards | scorecard_tools.py:46 | scorecard_kpis | SELECT (deduped, junk-filtered) |
| _scorecard_summary | scorecard_tools.py:111 | scorecard_kpis | SELECT (aggregated) |
| e_scorecard | scorecard_tools.py:164 | scorecard_kpis | INSERT |
| e_scorecard | scorecard_tools.py:219 | scorecard_kpis | UPDATE |
| e_scorecard | scorecard_tools.py:304 | scorecard_kpis | DELETE (admin only) |
| _initiatives | initiative_tools.py:47 | initiatives_details | SELECT (with RBAC) |
| e_initiative_progress | initiative_tools.py:114 | initiatives_details | UPDATE (JSON blob) |
| e_incident_status | incident_tools.py:44 | universal_incident | UPDATE (nested JSON) |
| _decisions | decision_tools.py:60 | decisions | SELECT (with RBAC) |
| e_decision | decision_tools.py:125 | decisions | INSERT |
| e_decision_status | decision_tools.py:219 | decisions | UPDATE (column + JSON) |

## L Prediction Tools (Registry)

tool_registry.py:52-172

oost-based prediction models are registered but invoked via a separate ML tool registry (not through the `[TOOL_CALL:...]` ism):

| | Model File | Description |
|---|---|---|
| ct_risk | risk_model.joblib | Residual risk score prediction |
| ct_revenue | revenue_model.joblib | Revenue attainment % |
| ct_attrition | attrion_model.joblib | Employee attrition probability |

| | |
|---|---|
| 2 | If present: load last 20 messages from `agent_messages` via `_load_history()` |
| 3 | If absent: start a fresh message list |
| 4 | Append the new user message and send the assembled `messages` to the LLM |
| 5 | After the LLM responds, create a conversation via `POST /conversations` if none existed |
| 6 | Save both user and assistant messages to `agent_messages` |
| 7 | Store an insight in `ai_memory`, then prune stale memories |

### ySQL Tables

| | Purpose | Key Columns |
|---|---|---|
| `_conversations` | Chat session metadata | `id`, `agent_name`, `user_id`, `org_id`, `title`, `created_at` |
| `_messages` | Individual chat messages | `id`, `conversation_id`, `role`, `content`, `created_at` |
| `mory` | Long-term insight storage | `id`, `user_id`, `org_id`, `agent_name`, `insight`, `confidence`, `access_count` |
| `ent_runs` | Per-call observability | `id`, `agent_name`, `provider`, `model`, `status`, `duration_ms`, `token_count` |

## tput Guardrails & Token Governance

### put Sanitization Pipeline

#### re 4 — Memory Lifecycle

| | |
|---|---|
| | **Store:** LLM response passed to `store_memory()` (ai/memory.py) |
| 2 | Filtered: trivial exchanges (greetings/small talk), responses < 40 chars, messages < 10 chars → skipped |
| 3 | Confidence scored 0.0-1.0 (base 0.4 + data-richness bonuses); rejected if < 0.3 |
| 4 | Deduplicated via first-80-chars comparison against existing `ai_memory` rows |
| 5 | **INSERT** into `ai_memory` (user_id, org_id, agent_name, insight, confidence, source) |
| 6 | **Retrieve:** SELECT user-specific memories ordered by confidence DESC, accessed_at DESC (limit 8) |
| 7 | Fallback: if fewer than 3 user-specific, supplement with org-wide memories |
| 8 | **Prune:** after each store, if count > 200 per agent, delete lowest access_count + confidence + oldest |

### tput Validation Pipeline

#### re 7 — Output Validation Pipeline

| | |
|---|---|
| | Raw LLM output from provider |
| 2 | Empty response (< 5 chars) → safe fallback message returned |
| 3 | Length > 50,000 chars → truncated with marker |
| | |